

Assignment 3: TCP Congestion Control Implementation

CS 422: Computer Networks, Fall 2026

Instructor: Vamsi Addanki

TAs: Youngsuk Kim, Xiao Luo, and Albert Vo

Due Date: October 21, 2026 @ 11:45 PM Eastern Time

Choose **one** of the following options to implement and evaluate the congestion control algorithm you designed in Assignment 2, Question 3(b). Explain how your implementation updates the congestion window using RTT, packet-loss estimates or a loss proxy, and throughput. Document its initial state, parameters, and any changes from your original pseudocode.

Option 1. Kernel or eBPF Implementation (100 points + bonus)

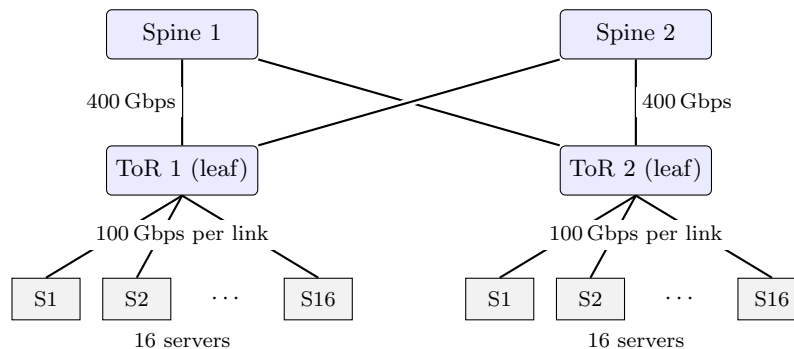
Write a kernel module or eBPF program implementing your congestion control algorithm. Select it on the data socket in your Python client from Assignment 2 and record the algorithm actually used. For each test, log congestion window, throughput, RTT, and a loss signal (e.g., 3 DUP ACKs, timeouts, or retransmission events) for 10 seconds.

Repeat the tests with CUBIC, Reno, and BBR across multiple runs. Use the same destinations, duration, and sampling interval for all four algorithms, and report the number of repetitions and test order. Repeat the time-series and scatter plots from Assignment 2, Question 2(b), for all four algorithms using the same representative destination. Clearly label the algorithms and use consistent axis scales. Include throughput and rtt aggregate statistics and explain the observed differences.

Bonus: This option is eligible for bonus points of +25 in the final-exam. For example, if a student has a score of 75 out of 100 in the final exam and also has 25 bonus points from this assignment, then the final exam score will be revised to 100 out of 100. Note that the final-exam score cannot exceed 100 though, and the bonus cannot compensate for other components in the grading scheme.

Option 2. ns-3 Implementation (100 points; no bonus)

Implement your algorithm from Assignment 2 in ns-3 within the TCP/IP stack. Evaluate your algorithm in the following setting:



Each ToR connects to both spines and to 16 servers (intermediate servers omitted).

All links are bidirectional, with a propagation delay of 500 ns in each direction.

- Create a leaf-spine topology with 2 ToR switches, 2 spine switches, and 16 servers per ToR (32 servers total). Model switches and servers as nodes. Connect each server to its ToR at 100 Gbps and each ToR to both spine switches at 400 Gbps. All links are bidirectional, with a propagation delay of 500 ns in each direction.

- From each server, launch one flow to every other server, excluding itself ($32 \times 31 = 992$ directed flows). You may use `BulkSendApplication` in ns-3. Set each flow size to 64 MB. All flows should start simultaneously.
- Log the completion time of each flow, measured from application start until the receiver has received all application bytes. Allow sufficient simulation time for completion and explicitly report any unfinished flows.
- Repeat the experiment with CUBIC, `TcpNewReno`, and BBR, keeping the topology, workload, routing, and buffers fixed.
- Plot the mean and 99th-percentile flow completion times for your algorithm, CUBIC, `TcpNewReno`, and BBR. State the percentile calculation method. Use `TcpNewReno` as the Reno baseline for this option. Explain the observed differences.

Choose and justify the buffer sizes at each switch, citing a source in relation to switch size and port bandwidth. State the queue model, buffer units, and whether limits apply per port or to a shared buffer. Document the routing configuration and how paths through the two spine switches are selected.

Report

Include a GitHub or GitLab repository link, with links to the relevant functions or line ranges. Embed plots and analysis in the report and identify the code that generates them. Brightspace submission should be a single PDF.

For Option 1, document the exact Linux kernel version, required features, and build and load instructions. Use a kernel compatible with your implementation. During grading, demonstrate that your module or eBPF program compiles and runs on your machine; we will review the code for correctness. The grading environment will not load your kernel module or eBPF program.

For Option 2, document the exact stable ns-3 release and any modifications or additional dependencies. Include a single script that runs all experiments and generates the plots so that we can test with the same version.

For either option, provide raw measurements, configuration files, dependencies, and reproduction instructions. AI tools are allowed. Briefly acknowledge how you used them, and be prepared to explain your implementation and design choices during evaluation.

Note: Grading is still based on the oral exam during PSO/office hours in the week of *Oct 26*.

Grace period: 3 days (72 Hours) grace period is allowed for submission. After the grace period, 25% off for every 24 hours late, rounded up.